

Strings

NOTE : What is a String in C language?

- A string is a sequence of characters terminated with a null character (`\0`).
- There is **no built-in type** in C for strings.
- Strings are represented using a **character array** or **character pointer**.

E.g.

```
char ch[] = "Hello";
```

Memory Layout : 'H' 'e' 'l' 'l' 'o' '\0'

`sizeof(ch)` => 6 (one extra for null characters)

NOTE : Strings vs Character Array

- A character array may or may not be a string.
- It becomes a string only if it terminates with a null character (`\0`).

E.g.

```
char ch1[5] = {'H', 'e', 'l', 'l', 'o'}; // it is just a character array
```

```
char ch2[10] = {'H', 'e', 'l', 'l', 'o'}; // it is treated as string, why?
```

The second array is treated as string because it has at least one null character (`\0`). Let's see the memory layout for second character array.,

`ch2[10]` => 'H' 'e' 'l' 'l' 'o' '\0' '\0' '\0' '\0' '\0'

Notice that the rest of the elements are automatically initialised by null character (\0).

NOTE : Ways to Initialise Strings
(READ CHARACTER POINTER NOTES)

NOTE : Difference between `char ch[ ]` and `char * ch`

Feature	<code>char ch[] = "Hi"</code>	<code>char* ch = "Hi"</code>
Storage	stack/ writable memory	Read only memory
Modifiable	Yes	No
Reassignable	No	Yes

NOTE : Size vs Length of a String

Size denotes the total memory required to store the string. Note that a string is terminated with a null character. Therefore (n+1) bytes are required to store a string of length n.

While Length of the String denotes the total number of characters before the first null character.

E.g.

```
char ch[ ] = "Hello";  
sizeof(ch) ; -> 6 bytes  
strlen(ch); -> 5
```

```
char ch2[10] = "Hi";  
sizeof(ch); -> 10 bytes
```

strlen(ch); -> 2

NOTE : Input and Output Operations on String

(A) Taking Input to a String

(a) Using gets()

`char * gets(char* str)`

- available in the `stdio.h` library
- In the parameter, it accepts a character pointer to the character array.
- It returns the same pointer as passed in the parameter. If any error is occurred, it returns **NULL**.

E.g.

```
char ch[20];  
gets(ch); // suppose input is "HELLO WORLD"  
printf("%s", ch); // Output : HELLO WORLD
```

- **Deprecated** : It does not check the buffer size, causing buffer overflow.

(b) Using scanf()

```
char ch[20];  
scanf("%s", ch); // suppose input is "HELLO WORLD"  
printf("%s", ch); // Output : HELLO WORLD
```

- However `scanf( )` is also not safe while taking input, because it does not check the size of an array in which we are taking the input. Thus, it may cause **buffer overflow**.
- `char ch[5];`

```
scanf("%s", ch); // Suppose the input is :  
"HELLOWORLD"
```

This will cause **buffer overflow**, as `scanf()` doesn't check the size of the array.

What could be the solution then?

1.) Using some counter value, take only that many inputs as much as the size of the array allows.

2.) Using **Width Specifier** with `scanf()`

```
char ch[10];  
scanf("%9s", ch); // one element is left for null  
character
```

We have already discussed in the "Character Pointer" notes on how to take whitespaces as input using `scanf()`. Go through that note.

(c) Using `fgets()` - (Recommended)

```
char * fgets(char* str, int size, FILE* stream)
```

str -> pointer to the buffer where we want to take the input

size -> maximum number of characters to read including null character (`\0`)

stream -> input source (`stdin`, file pointer, etc)

Returns **str** on **success**, else returns **NULL** or **EOF**(end of file) on **error**.

E.g.

```
char ch[5];  
fgets(ch, 5, stdin); // Suppose we give input "HEY"  
printf("%s", ch); // Output : HEY
```

- `fgets()` stops taking input after reading (`size-1`) characters or when newline (`\n`) is encountered.
- `char ch[10];`
`fgets(ch, 10, stdin); "Hello\n"`
`fgets()` reads 'H', 'e', 'l', 'l', 'o', '\n', '\0', '\0', '\0', '\0'

Functions	Stops at Newline	Stores Newline	Buffer Safe
<code>scanf()</code>	Yes	No	No
<code>gets()</code>	Yes	No	No
<code>fgets()</code>	Yes	Yes	Yes

NOTE : Common String Library Functions ([string.h](#))

- These functions are available in the library - [string.h](#)

Let's look at some of these functions one by one.

1. `strlen()` : `size_t` `strlen(const char* str)`

str : pointer to null terminated string

const char* : `const` is used so that the `strlen()` function does not modify the original string

Return Type : `size_t` (alias for unsigned integer) -> returns the length of the string (excluding null character)

Q. What happens when we pass a non-null terminated string?

A. Undefined behaviour.

Note : `sizeof(str) != strlen(str)`

2. `strcpy( ) : char * strcpy(char* dest, const char* src)`

dest : destination buffer (must be large enough)

src : pointer to source string (must be null terminated)

const char* src : const so that the function could not modify the original string passed to the function

Returns : dest

Note that the destination buffer must be of enough size. There is no bound checking while copying the string, so buffer overflow is possible.

Q. What could happen if the dest buffer size is not large enough to store the src string?

A. Let's look at the following example

```
char dest[4];  
char src[ ] = "HELLO";  
strcpy(dest, src);
```

dest[0] -> 'H'

dest[1] -> 'E'

dest[2] -> 'L'

dest[3] -> 'L'

dest[4] -> 'O' // out of bounds, unauthorized access to memory

dest[5] -> '\0' // out of bounds, unauthorized access to memory

These are some possibilities which could happen:

- a) Appear to work
- b) Print garbage
- c) Segmentation fault
- d) Corrupt nearby variables or may corrupt other programs

There is no guaranteed behaviour.

3.) strncpy() : `char *strncpy(char *dest, const char *src, size_t n)`

dest : destination buffer

src : pointer to source string

n : maximum number of characters to copy

Return value : dest

- Copies at most n characters from src to dest
- If $n \leq \text{length of src string}$, then the remaining elements are filled with `'\0'`
- If $n > \text{length of src string}$, then no `'\0'` is added.

```
char dest[10];  
strncpy(dest, "Hi", 10);
```

dest => 'H', 'i', '\0', ... , '\0'

Now consider this :

```
char dest[5];  
strncpy(dest, "HELLO", 5);  
printf("%s", dest); // Undefined Behavior
```

dest => 'H', 'E', 'L', 'L', 'O' (No null terminator added, not a valid string)

Best Practice to use the following format :

```
strncpy(dest, src, sizeof(dest) - 1);  
dest[sizeof(dest) - 1] = '\0';
```

4.) `char* strcat(char* dest, const char* src)`

dest : destination string (must be null terminated and large enough)

src : source string to be concatenated with the destination string(null terminated)

const char* : const so that the source string doesn't get modified inside the function

Return : dest

- Appends src after the existing content of dest
- Starts appending src starting at the \0 of dest
- Copies characters from src including '\0'

```
char a[20] = "Hello";  
char b[] = "World";
```

```
strcat(a, b);  
printf("%s", a); // HelloWorld
```

- dest must have enough free space
- No bounds checking -> may lead to buffer overflow
- dest must already contain '\0'
- Unsafe example :

```
char a[5] = "Hi";  
strcat(a, "Hello"); // Undefined behavior
```

5.) strcmp() : **int** strcmp(**const char** *str1, **const char** *str2);

str1 : pointer to string 1

str2 : pointer to string 2

Return :

0 if both strings are same

(< 0) if str1 is smaller than str2

(> 0) if str1 is greater than str2

NOTE : DO NOT USE (==) operator to compare strings!!!

- Comparison is case-sensitive, based on ASCII values of the character.

NOTE : Modifying Strings

We have already discussed this while studying “Ways to Initialise Strings” in the “Character Pointer” notes. Please refer to that note!

NOTE : Array of Strings

There are many ways to represent an array of strings. Just like we studied 2D arrays of integers, this is the same concept, but with characters.

1.) 2D Character Array

```
char str[2][10] = {"Saurabh", "Kumar"};
```

[2] -> denotes the number of strings (Analogy : number of 1D arrays in 2D array)

[10] -> denotes the length of each string (Analogy : size of each 1D array in 2D array)

2.) Array of Character Pointers

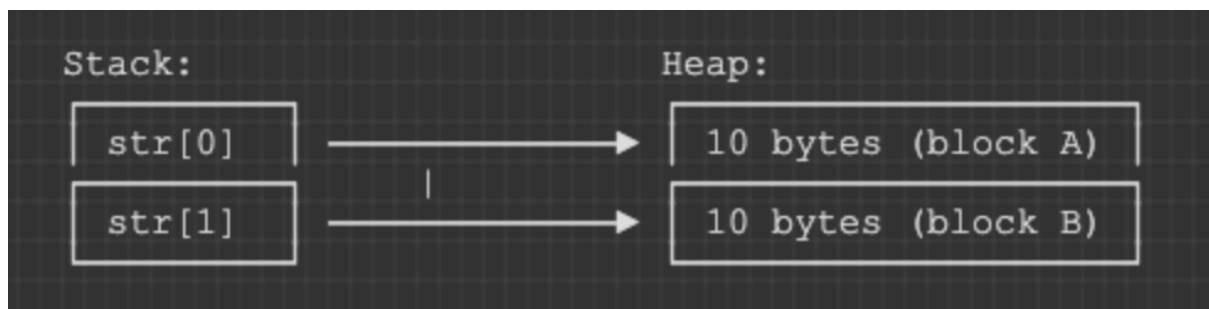
```
char* str[2] = {"Saurabh", "Kumar"}; // Initialisation while declaration
```

OR

```
char* str[2];
```

```
// dynamic memory allocation
for(int i = 0; i < 2; i++){
    str[i] = (char *)malloc(10*sizeof(char));
}
```

```
// explicit input by the user
for(int i = 0; i < 2; i++){
    fgets(str[i], 10, stdin);
}
```



NOTE : Array of Strings

Well, we now very well know that a string is an array of characters terminated with a null character.

So, we can treat a string as a 1D character array, after all it is in fact an array.

So, what could be an array of arrays?
Yeah, you got it right! It's a 2D array.

Let's see Array of Strings.

a) 2D Array of Characters

```
char names[3][10] = {"Saurabh", "Kumar", "IT Dept."};
```

b) Array of Character Pointers

```
char * names[2] = {"Saurabh", "Kumar"};
```

Each element of the array "names" is a character pointer to some string literals ("Saurabh", "Kumar", and "IT Dept.").

NOTE : Passing Strings to Functions

If we want to modify or access the string, we would need the base address of the string. Since a string is just a character array, its name itself represents the base address(address of the whole array, value). Thus we need to pass the address of the string to any function. And to catch that address, we would need a pointer(character pointer).

Let's see some examples

```
void print( char str[ ] ) {  
    printf("%s", str);  
}
```

Note that char str[] in the parameter accepts the address of the character array. It is equivalent to char *str.

```
void print(char *str)
```

NOTE : Dynamic Memory and Strings

Just like integer arrays, we can allocate dynamic memory to character arrays.

If we can make character arrays dynamically, then we can easily convert those character arrays to strings by putting '\0' at the end.

Thus, a string can be created dynamically, which will be stored in the HEAP area of the program.

```
char *str = (char *)malloc(10*sizeof(char));  
fgets(str, 10, stdin);  
free(ptr);
```

**IF YOU ARE READING THIS MESSAGE, THEN IT MEANS
YOU HAVE FOLLOWED THIS NOTE THOROUGHLY.**

KEEP IT UP !!!